Array programming

In computer science, **array programming** refers to solutions that allow the application of operations to an entire set of values at once. Such solutions are commonly used in scientific and engineering settings.

Modern programming languages that support array programming (also known as vector or multidimensional languages) have been engineered specifically to generalize operations on scalars to apply transparently to vectors, matrices, and higher-dimensional arrays. These include APL, J, Fortran, MATLAB, Analytica, Octave, PL/I, R, Cilk Plus, Julia, Perl Data Language (PDL) and Raku. In these languages, an operation that operates on entire arrays can be called a *vectorized* operation,[1] regardless of whether it is executed on a vector processor, which implements vector instructions. Array programming primitives concisely express broad ideas about data manipulation. The level of concision can be dramatic in certain cases: it is not uncommon to find array programming language one-liners that require several pages of object-oriented code.

Concepts of array

The fundamental idea behind array programming is that operations apply at once to an entire set of values. This makes it a high-level programming model as it allows the programmer to think and operate on whole aggregates of data, without having to resort to explicit loops of individual scalar operations.

Kenneth E. Iverson described the rationale behind array programming (actually referring to APL) as follows:[2]

> most programming languages are decidedly inferior to mathematical notation and are little used as tools of thought in ways that would be considered significant by, say, an applied mathematician.
>
> The thesis is that the advantages of executability and universality found in programming languages can be effectively combined, in a single coherent language, with the advantages offered by mathematical notation. it is important to distinguish the difficulty of describing and of learning a piece of notation from the difficulty of mastering its implications. For example, learning the rules for computing a matrix product is easy, but a mastery of its implications (such as its associativity, its distributivity over addition, and its ability to represent linear functions and geometric operations) is a different and much more difficult matter.
>
> Indeed, the very suggestiveness of a notation may make it seem harder to learn because of the many properties it suggests for explorations.
>
> [...]
>
> Users of computers and programming languages are often concerned primarily with the efficiency of execution of algorithms, and might, therefore, summarily dismiss many of the algorithms presented here. Such dismissal would be short-sighted since a clear statement of an

algorithm can usually be used as a basis from which one may easily derive a more efficient algorithm.

The basis behind array programming and thinking is to find and exploit the properties of data where individual elements are similar or adjacent. Unlike object orientation which implicitly breaks down data to its constituent parts (or scalar quantities), array orientation looks to group data and apply a uniform handling.

Function rank is an important concept to array programming languages in general, by analogy to tensor rank in mathematics: functions that operate on data may be classified by the number of dimensions they act on. Ordinary multiplication, for example, is a scalar ranked function because it operates on zero-dimensional data (individual numbers). The cross product operation is an example of a vector rank function because it operates on vectors, not scalars. Matrix multiplication is an example of a 2-rank function, because it operates on 2-dimensional objects (matrices). Collapse operators reduce the dimensionality of an input data array by one or more dimensions. For example, summing over elements collapses the input array by 1 dimension.

Uses

Array programming is very well suited to implicit parallelization; a topic of much research nowadays. Further, Intel and compatible CPUs developed and produced after 1997 contained various instruction set extensions, starting from MMX and continuing through SSSE3 and 3DNow!, which include rudimentary SIMD array capabilities. This has continued into the 2020s with instruction sets such as AVX-512, making modern CPUs sophisticated vector processors. Array processing is distinct from parallel processing in that one physical processor performs operations on a group of items simultaneously while parallel processing aims to split a larger problem into smaller ones (MIMD) to be solved piecemeal by numerous processors. Processors with multiple cores and GPUs with thousands of general computing cores are common as of 2023.

Languages

The canonical examples of array programming languages are Fortran, APL, and J. Others include: A+, Analytica, Chapel, IDL, Julia, K, Klong, Q, MATLAB, GNU Octave, Scilab, FreeMat, Perl Data Language (PDL), R, Raku, S-Lang, SAC, Nial, ZPL, Futhark, and TI-BASIC.

Scalar languages

In scalar languages such as C and Pascal, operations apply only to single values, so *a*+*b* expresses the addition of two numbers. In such languages, adding one array to another requires indexing and looping, the coding of which is tedious.

```
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        a[i][j] += b[i][j];
    }
}
```

In array-based languages, for example in Fortran, the nested for-loop above can be written in array-format in one line,

```
a = a + b
```

or alternatively, to emphasize the array nature of the objects,

```
a(:,:) = a(:,:) + b(:,:)
```

While scalar languages like C do not have native array programming elements as part of the language proper, this does not mean programs written in these languages never take advantage of the underlying techniques of vectorization (i.e., utilizing a CPU's vector-based instructions if it has them or by using multiple CPU cores). Some C compilers like GCC at some optimization levels detect and vectorize sections of code that its heuristics determine would benefit from it. Another approach is given by the OpenMP API, which allows one to parallelize applicable sections of code by taking advantage of multiple CPU cores.

Array languages

In array languages, operations are generalized to apply to both scalars and arrays. Thus, *a+b* expresses the sum of two scalars if *a* and *b* are scalars, or the sum of two arrays if they are arrays.

An array language simplifies programming but possibly at a cost known as the *abstraction penalty*.[3][4][5] Because the additions are performed in isolation from the rest of the coding, they may not produce the optimally most efficient code. (For example, additions of other elements of the same array may be subsequently encountered during the same execution, causing unnecessary repeated lookups.) Even the most sophisticated optimizing compiler would have an extremely hard time amalgamating two or more apparently disparate functions which might appear in different program sections or sub-routines, even though a programmer could do this easily, aggregating sums on the same pass over the array to minimize overhead).

Ada

The previous C code would become the following in the Ada language,[6] which supports array-programming syntax.

```
A := A + B;
```

APL

APL uses single character Unicode symbols with no syntactic sugar.

```
A ← A + B
```

This operation works on arrays of any rank (including rank 0), and on a scalar and an array. Dyalog APL extends the original language with augmented assignments:

```
A +← B
```

Analytica

Analytica provides the same economy of expression as Ada.

```
A := A + B;
```

BASIC

Dartmouth BASIC had MAT statements for matrix and array manipulation in its third edition (1966).

```
DIM A(4),B(4),C(4)
MAT A = 1
MAT B = 2 * A
MAT C = A + B
MAT PRINT A,B,C
```

Mata

Stata's matrix programming language Mata supports array programming. Below, we illustrate addition, multiplication, addition of a matrix and a scalar, element by element multiplication, subscripting, and one of Mata's many inverse matrix functions.

```
. mata:

: A = (1,2,3) \(4,5,6)

: A
       1   2   3
    +-------------+
  1 |  1   2   3  |
  2 |  4   5   6  |
    +-------------+

: B = (2..4) \(1..3)

: B
       1   2   3
    +-------------+
  1 |  2   3   4  |
  2 |  1   2   3  |
    +-------------+

: C = J(3,2,1)           // A 3 by 2 matrix of ones

: C
       1   2
    +---------+
  1 |  1   1  |
  2 |  1   1  |
  3 |  1   1  |
    +---------+

: D = A + B

: D
       1   2   3
    +-------------+
  1 |  3   5   7  |
  2 |  5   7   9  |
```

```
    +-------------+

: E = A*C

: E
        1    2
    +-----------+
  1 |   6    6  |
  2 |  15   15  |
    +-----------+

: F = A:*B

: F
        1    2    3
    +----------------+
  1 |   2    6   12  |
  2 |   4   10   18  |
    +----------------+

: G = E :+ 3

: G
        1    2
    +-----------+
  1 |   9    9  |
  2 |  18   18  |
    +-----------+

: H = F[(2\1), (1, 2)]    // Subscripting to get a submatrix of F and

:                         // switch row 1 and 2
: H
        1    2
    +-----------+
  1 |   4   10  |
  2 |   2    6  |
    +-----------+

: I = invsym(F'*F)        // Generalized inverse (F*F^(-1)F=F) of a

:                         // symmetric positive semi-definite matrix
: I
[symmetric]
                 1             2             3
    +-------------------------------------------+
  1 |            0                               |
  2 |            0          3.25                 |
  3 |            0         -1.75   .9444444444   |
    +-------------------------------------------+

: end
```

MATLAB

The implementation in MATLAB allows the same economy allowed by using the Fortran language.

```
A = A + B;
```

A variant of the MATLAB language is the GNU Octave language, which extends the original language with augmented assignments:

```
A += B;
```

Both MATLAB and GNU Octave natively support linear algebra operations such as matrix multiplication, matrix inversion, and the numerical solution of system of linear equations, even using the Moore–Penrose pseudoinverse.[7][8]

The Nial example of the inner product of two arrays can be implemented using the native matrix multiplication operator. If `a` is a row vector of size [1 n] and `b` is a corresponding column vector of size [n 1].

```
a * b;
```

By contrast, the entrywise product is implemented as:

```
a .* b;
```

The inner product between two matrices having the same number of elements can be implemented with the auxiliary operator `(:)`, which reshapes a given matrix into a column vector, and the transpose operator `'`:

```
A(:)' * B(:);
```

rasql

The rasdaman query language is a database-oriented array-programming language. For example, two arrays could be added with the following query:

```
SELECT A + B
FROM   A, B
```

R

The R language supports array paradigm by default. The following example illustrates a process of multiplication of two matrices followed by an addition of a scalar (which is, in fact, a one-element vector) and a vector:

```
> A <- matrix(1:6, nrow=2)                          # !!this has nrow=2 ... and A has 2 rows
> A
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
> B <- t( matrix(6:1, nrow=2) )  # t() is a transpose operator                          !!this has
nrow=2 ... and B has 3 rows --- a clear contradiction to the definition of A
> B
     [,1] [,2]
[1,]    6    5
[2,]    4    3
[3,]    2    1
> C <- A %*% B
> C
     [,1] [,2]
[1,]   28   19
[2,]   40   28
> D <- C + 1
> D
     [,1] [,2]
[1,]   29   20
```

```
[2,]   41   29
> D + c(1, 1)  # c() creates a vector
     [,1] [,2]
[1,]   30   21
[2,]   42   30
```

Raku

Raku supports the array paradigm via its Metaoperators.[9] The following example demonstrates the addition of arrays @a and @b using the Hyper-operator in conjunction with the plus operator.

```
[0] > my @a = [[1,1],[2,2],[3,3]];
[[1 1] [2 2] [3 3]]

[1] > my @b = [[4,4],[5,5],[6,6]];
[[4 4] [5 5] [6 6]]

[2] > @a »+« @b;
[[5 5] [7 7] [9 9]]
```

Mathematical reasoning and language notation

The matrix left-division operator concisely expresses some semantic properties of matrices. As in the scalar equivalent, if the (determinant of the) coefficient (matrix) `A` is not null then it is possible to solve the (vectorial) equation `A * x = b` by left-multiplying both sides by the inverse of `A`: A^{-1} (in both MATLAB and GNU Octave languages: `A^-1`). The following mathematical statements hold when `A` is a full rank square matrix:

```
A^-1 *(A * x)==A^-1 * (b)
(A^-1 * A)* x ==A^-1 * b      (matrix-multiplication associativity)
x = A^-1 * b
```

where `==` is the equivalence relational operator. The previous statements are also valid MATLAB expressions if the third one is executed before the others (numerical comparisons may be false because of round-off errors).

If the system is overdetermined – so that `A` has more rows than columns – the pseudoinverse A^{+} (in MATLAB and GNU Octave languages: `pinv(A)`) can replace the inverse A^{-1}, as follows:

```
pinv(A) *(A * x)==pinv(A) * (b)
(pinv(A) * A)* x ==pinv(A) * b      (matrix-multiplication associativity)
x = pinv(A) * b
```

However, these solutions are neither the most concise ones (e.g. still remains the need to notationally differentiate overdetermined systems) nor the most computationally efficient. The latter point is easy to understand when considering again the scalar equivalent `a * x = b`, for which the solution `x = a^-1 * b` would require two operations instead of the more efficient `x = b / a`. The problem is that generally matrix multiplications are not commutative as the extension of the scalar solution to the matrix case would require:

```
(a * x)/ a ==b / a
```

```
(x * a)/ a ==b / a      (commutativity does not hold for matrices!)
x * (a / a)==b / a      (associativity also holds for matrices)
x = b / a
```

The MATLAB language introduces the left-division operator `\` to maintain the essential part of the analogy with the scalar case, therefore simplifying the mathematical reasoning and preserving the conciseness:

```
A \ (A * x)==A \ b
(A \ A)* x ==A \ b      (associativity also holds for matrices, commutativity is no more required)
x = A \ b
```

This is not only an example of terse array programming from the coding point of view but also from the computational efficiency perspective, which in several array programming languages benefits from quite efficient linear algebra libraries such as ATLAS or LAPACK.[10]

Returning to the previous quotation of Iverson, the rationale behind it should now be evident:

> it is important to distinguish the difficulty of describing and of learning a piece of notation from the difficulty of mastering its implications. For example, learning the rules for computing a matrix product is easy, but a mastery of its implications (such as its associativity, its distributivity over addition, and its ability to represent linear functions and geometric operations) is a different and much more difficult matter. Indeed, the very suggestiveness of a notation may make it seem harder to learn because of the many properties it suggests for explorations.

Third-party libraries

The use of specialized and efficient libraries to provide more terse abstractions is also common in other programming languages. In C++ several linear algebra libraries exploit the language's ability to overload operators. In some cases a very terse abstraction in those languages is explicitly influenced by the array programming paradigm, as the NumPy extension library to Python, Armadillo and Blitz++ libraries do.[11][12]

See also

- Array slicing
- List of array programming languages
- Automatic vectorization

References

1. Stéfan van der Walt; S. Chris Colbert & Gaël Varoquaux (2011). "The NumPy array: a structure for efficient numerical computation". *Computing in Science and Engineering*. **13**

(2). IEEE: 22–30. arXiv:1102.1523 (https://arxiv.org/abs/1102.1523). Bibcode:2011CSE....13b..22V (https://ui.adsabs.harvard.edu/abs/2011CSE....13b..22V). doi:10.1109/mcse.2011.37 (https://doi.org/10.1109%2Fmcse.2011.37). S2CID 16907816 (https://api.semanticscholar.org/CorpusID:16907816).
2. Iverson, K. E. (1980). "Notation as a Tool of Thought" (https://doi.org/10.1145%2F358896.358899). *Communications of the ACM*. **23** (8): 444–465. doi:10.1145/358896.358899 (https://doi.org/10.1145%2F358896.358899).
3. Surana P (2006). *Meta-Compilation of Language Abstractions* (https://dl.acm.org/doi/book/10.5555/1195058) (Thesis).
4. Kuketayev. "The Data Abstraction Penalty (DAP) Benchmark for Small Objects in Java" (https://web.archive.org/web/20090111091710/http://www.adtmag.com/joop/article.aspx?id=4597). Archived from the original (http://www.adtmag.com/joop/article.aspx?id=4597) on 2009-01-11. Retrieved 2008-03-17.
5. Chatzigeorgiou; Stephanides (2002). "Evaluating Performance and Power Of Object-Oriented Vs. Procedural Programming Languages". In Blieberger; Strohmeier (eds.). *Proceedings - 7th International Conference on Reliable Software Technologies - Ada-Europe'2002* (https://books.google.com/books?id=QMalP1P2kAMC&q=%22abstraction+penalty%22). Springer. p. 367. ISBN 978-3-540-43784-0.
6. Ada Reference Manual (http://www.adaic.org/standards/05rm/html/RM-TTL.html): G.3.1 Real Vectors and Matrices (http://www.adaic.org/resources/add_content/standards/05rm/html/RM-G-3-1.html)
7. "GNU Octave Manual. Arithmetic Operators" (https://www.gnu.org/software/octave/doc/interpreter/Arithmetic-Ops.html). Retrieved 2011-03-19.
8. "MATLAB documentation. Arithmetic Operators" (https://web.archive.org/web/20100907074906/http://www.mathworks.com/help/techdoc/ref/arithmeticoperators.html). Archived from the original (http://www.mathworks.com/help/techdoc/ref/arithmeticoperators.html) on 2010-09-07. Retrieved 2011-03-19.
9. "Metaoperators section of Raku Operator documentation" (https://docs.raku.org/language/operators#Metaoperators).
10. "GNU Octave Manual. Appendix G Installing Octave" (https://www.gnu.org/software/octave/doc/interpreter/Installation.html). Retrieved 2011-03-19.
11. "Reference for Armadillo 1.1.8. Examples of Matlab/Octave syntax and conceptually corresponding Armadillo syntax" (https://arma.sourceforge.net/docs.html#syntax). Retrieved 2011-03-19.
12. "Blitz++ User's Guide. 3. Array Expressions" (https://web.archive.org/web/20110323013142/http://www.oonumerics.org/blitz/docs/blitz_3.html#SEC80). Archived from the original (http://www.oonumerics.org/blitz/docs/blitz_3.html#SEC80) on 2011-03-23. Retrieved 2011-03-19.

External links

- "No stinking loops" programming (http://www.nsl.com/)
- Discovering Array Languages (https://web.archive.org/web/20110227013846/http://www.vector.org.uk/archive/v223/smill222.htm)
- "Types of Arrays" programming (http://www.zareenacademy.com/)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Array_programming&oldid=1346731840"